MOBILE APPLICATION AND DEVELOPMENT · WEEK 3–4

Manifest, Permissions & UI Layouts

AndroidManifest.xml · Runtime permissions · Views and ViewGroups · LinearLayout · RelativeLayout

Ekramul Haque Amin · Lecturer, AI & Data Science · Dhaka International University

TODAY'S ROADMAP

What We Will Cover

1. Recap: Receivers and Providers
2. The manifest, permissions and project structure
3. Views, ViewGroups and how Android lays out a screen
4. LinearLayout
5. RelativeLayout

LAST CLASS

Recap: Receivers and Providers

Broadcast Receiver
Reacts to broadcasts. Register dynamically and unregister in onStop().

Content Provider
Shares data through content URIs and a ContentResolver.

Today
All four components are declared in one file: the manifest.

PART 3 — ANDROIDMANIFEST.XML

What Is the Manifest?

The manifest is your app's ID card for the system. Before running any of your code, Android reads it to learn what the app contains and what it is allowed to do.

Components
Every Activity, Service, Receiver and Provider

Permissions
What the app asks the user to allow

Requirements
Hardware and software features needed

Identity
Icon, label and theme of the app

THE SKELETON

Anatomy of a Manifest

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-feature android:name="android.hardware.camera"
                  android:required="false" />
    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.MyApp">
        <activity android:name=".MainActivity" android:exported="true" />
    </application>
</manifest>
```

THE TAGS YOU WILL MEET

Key Manifest Elements

Element	Purpose	Example
`<uses-permission>`	Asks for a permission	INTERNET, CAMERA
`<uses-feature>`	Declares hardware needs	camera, GPS
`<application>`	App-wide settings	icon, label, theme
`<activity> <service>`	Declare components	one tag each
`<receiver> <provider>`	Declare the other two	same idea
`<intent-filter>`	What a component handles	action, category, data

TELLING ANDROID WHAT YOU CAN HANDLE

Intent Filters

```
<activity android:name=".ShareActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:mimeType="text/plain" />
    </intent-filter>
</activity>
```

An implicit Intent matches a component when its action, category and data all fit the filter.

WHO MAY START THIS COMPONENT?

The exported Attribute

exported = true
Other apps can start this component. Needed for the launcher Activity.

exported = false
Only your own app can start it. The safe default.

Apps targeting Android 12 (API 31) or higher must set android:exported on every component that has an intent-filter.

THREE KINDS

Permission Types

Normal
Granted automatically at install.
Example: INTERNET.

Dangerous (runtime)
The user must approve while the app runs. Examples: CAMERA, READ_CONTACTS, LOCATION.

Special
Granted from a system settings screen.
Example: drawing over other apps.

Dangerous permissions must be both listed in the manifest and requested at runtime.

ASK AT THE MOMENT YOU NEED IT

Requesting a Permission at Runtime

```
if (ContextCompat.checkSelfPermission(this, Manifest.permission.READ_CONTACTS)
        ≠ PackageManager.PERMISSION_GRANTED) {
    ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.READ_CONTACTS}, 100);
}
@Override
public void onRequestPermissionsResult(int code, String[] perms, int[] results) {
    super.onRequestPermissionsResult(code, perms, results);
    if (code == 100 && results[0] == PackageManager.PERMISSION_GRANTED) loadContacts();
}
```

WHERE THINGS LIVE

Android Studio Project Structure

manifests
AndroidManifest.xml, the file we just studied

java
Your source code, such as MainActivity

res
layout, drawable, mipmap and values (strings, colors, themes)

Gradle Scripts
build.gradle and settings.gradle control the build

NOT EVERYTHING IS IN THE MANIFEST

Module build.gradle Essentials

```
android {
    namespace 'com.example.myapp'
    compileSdk 34
    defaultConfig {
        applicationId "com.example.myapp"
        minSdk 24
        targetSdk 34
        versionCode 1
        versionName "1.0"
    }
}
```

minSdk is the oldest Android you support. targetSdk is the version you have tested against. Numbers here are examples.

LEARN FROM THESE NOW

Common Manifest Mistakes

Missing component entry
An undeclared Activity crashes with ActivityNotFoundException

Permission in manifest only
Dangerous permissions also need a runtime request

Missing exported
Components with an intent filter must set it on Android 12+

SDK numbers scattered
Keep minSdk and targetSdk in build.gradle only

PART 3 — VIEWS AND LAYOUTS

What Is a View?

Everything on an Android screen is a View: a rectangle that draws itself and handles touches. TextView, Button, EditText and ImageView are all Views. A ViewGroup is a special View that holds other Views and decides where they go.

View
A single widget: TextView, Button, ImageView

ViewGroup
A container: LinearLayout, RelativeLayout, ConstraintLayout

Layout
Another name for a ViewGroup that arranges its children

A TREE OF VIEWS

The View Hierarchy

```
LinearLayout (ViewGroup, the root)
    |-- TextView "Welcome"
    |-- LinearLayout (ViewGroup, nested)
    | |-- EditText
    | |-- Button
    |-- ImageView
```

Your activity_main.xml file is this tree written in XML. setContentView() inflates it into real objects.

WHAT ANDROID DOES FOR EVERY SCREEN

How a Layout Is Drawn

1. Measure
Each parent asks its children how big they want to be.

2. Layout
The parent assigns every child a position and a size.

3. Draw
Each View paints itself on the screen.

Deeply nested layouts repeat these passes many times. Keep hierarchies shallow.

PICK THE RIGHT CONTAINER

Common Layout Types

Layout	How it arranges children	Good for
LinearLayout	In one row or one column	Forms, button rows
RelativeLayout	Relative to each other or the parent	Aligned or overlapping items
ConstraintLayout	Flat set of constraints	Complex screens (next class)
FrameLayout	Stacked on top of each other	A single image or fragment

EVERY VIEW NEEDS BOTH

layout_width and layout_height

match_parent
Fill all the space the parent offers

wrap_content
Be only as big as the content needs

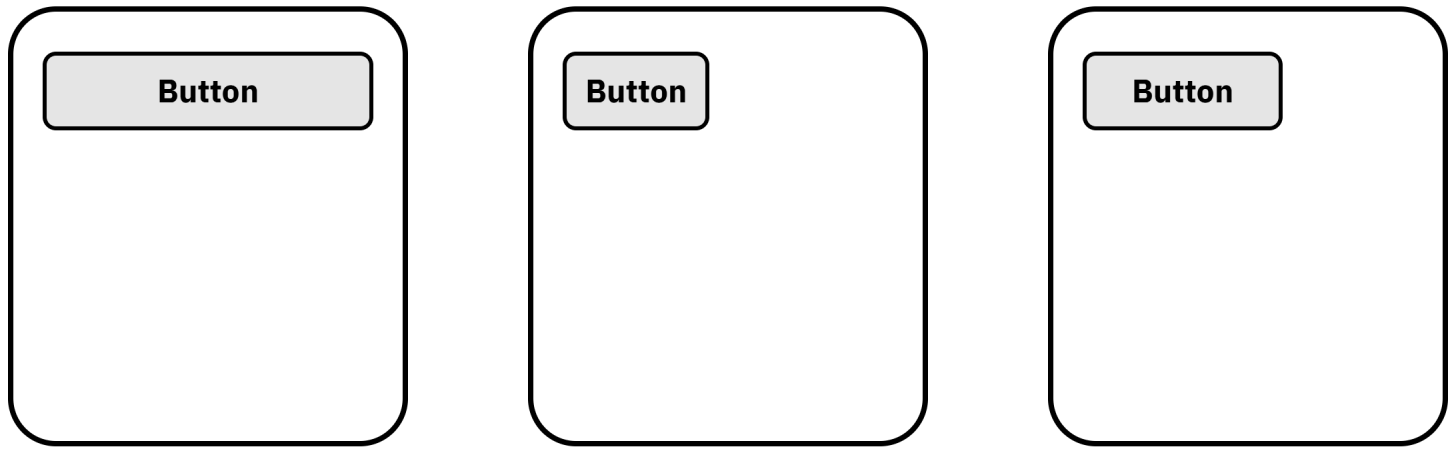
Fixed size
An exact value such as 200dp

```
android:layout_width="match_parent"
android:layout_height="wrap_content"
```

Leaving out either attribute is a build error.

WIDTH ON A PHONE SCREEN

match_parent, wrap_content and dp



layout_width="match_parent"

layout_width="wrap_content"

layout_width="160dp"

The Button is the same view in all three phones. Only the width rule changes.

UNITS THAT SURVIVE EVERY PHONE

dp, sp and px

dp
Density-independent pixels. Use for sizes, margins and padding.

sp
Like dp, but also follows the user's font-size setting. Use for text.

px
Real screen pixels. Avoid: the same value looks different on every phone.

```
android:layout_width="200dp"
android:textSize="18sp"
```

SPACE INSIDE AND OUTSIDE

Padding vs. Margin

Padding
Space inside a view, between its edge and its content.

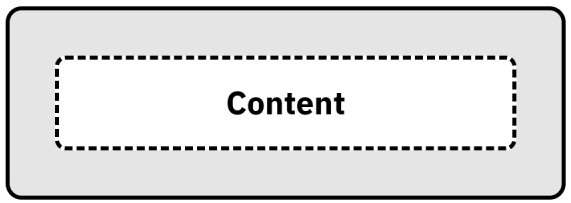
Margin
Space outside a view, between it and its neighbours.

```
android:padding="16dp" ←!— inside this view —→
android:layout_marginTop="8dp" ←!— outside this view —→
```

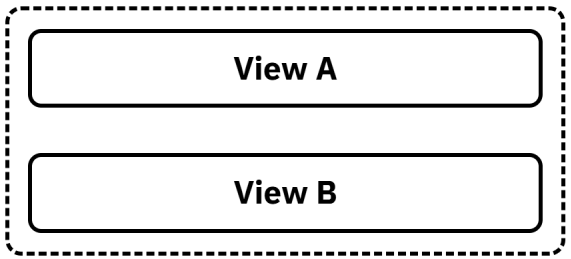
Margins are named layout_margin... because the parent layout applies them.

SEEING THE SPACE

Padding and Margin in Pictures



Padding: the grey band is space inside the view, around its content



Margin: the gap between A and B is space outside both views

Padding pushes content in. Margin pushes neighbours away.

TWO ATTRIBUTES, TWO JOBS

gravity vs. layout_gravity

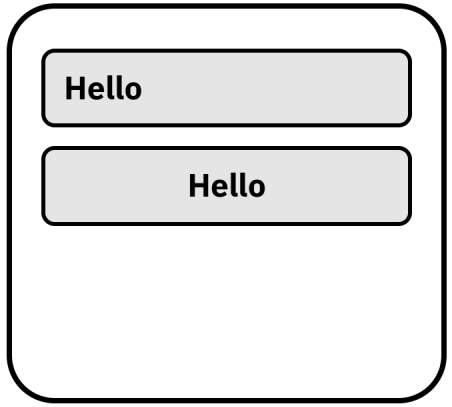
android:gravity
Aligns the content inside this view. Example: centre the text in a Button.

android:layout_gravity
Aligns this view inside its parent. Example: centre the Button in the layout.

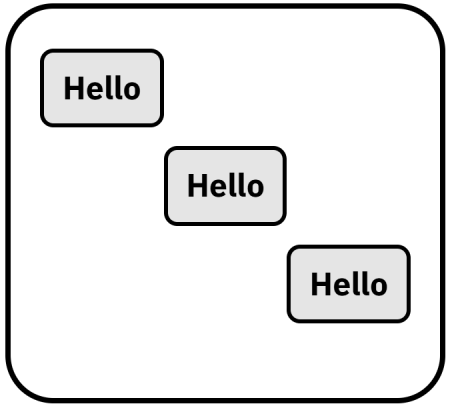
```
android:gravity="center" <!— content inside —>
android:layout_gravity="center" <!— view inside parent —>
```

CONTENT VS. POSITION

gravity and layout_gravity



android:gravity moves the text inside each wide TextView: start, then center



android:layout_gravity moves each narrow view inside the parent: start, center, end

PART 4 — LINEARLAYOUT

One Row or One Column

A LinearLayout lines its children up in a single direction. Set android:orientation to vertical for a column or horizontal for a row. It is the simplest layout and the right one for forms and button rows.

vertical
Children stacked top to bottom

horizontal
Children placed left to right

Order
Children appear in the same order as in the XML

A VERTICAL STACK

LinearLayout in XML

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="First" />
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Second" />
</LinearLayout>
```

SHARING LEFTOVER SPACE

layout_weight

```
<LinearLayout android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <Button android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1" android:text="Cancel" />
    <Button android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2" android:text="Save" />
</LinearLayout>
```

Cancel gets 1 share and Save gets 2 shares of the width. Use 0dp for the sized direction when using weights.

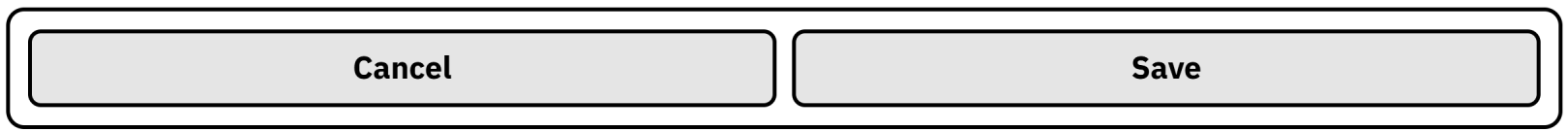
WHAT THE WEIGHTS DO

layout_weight in Pictures

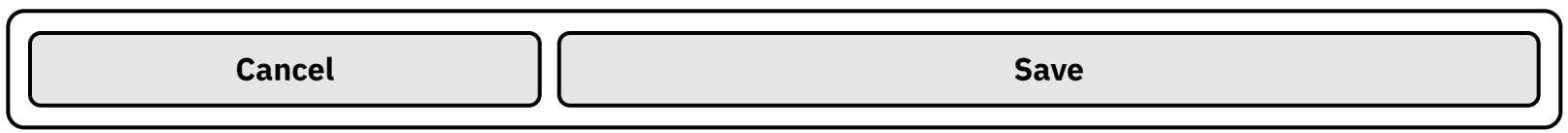
`no weights, wrap_content`



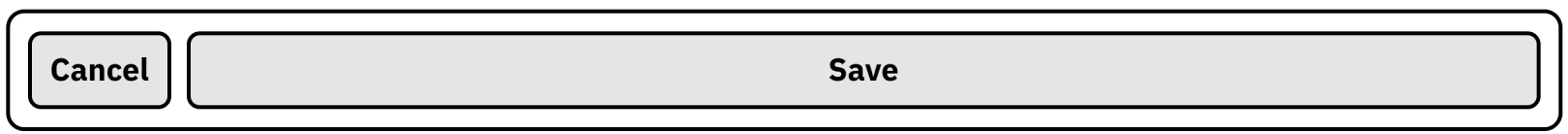
`weight 1 : 1`



`weight 1 : 2`



`weight 0 : 1 (Cancel wraps)`



Each row is one horizontal LinearLayout. With weights, leftover space is shared in proportion.

A REAL SCREEN

Login Form with LinearLayout

```
<LinearLayout ... android:orientation="vertical"
    android:padding="24dp">
    <EditText android:id="@+id/email"
        android:hint="Email" ... />
    <EditText android:id="@+id/password"
        android:hint="Password"
        android:inputType="textPassword" ... />
    <Button android:id="@+id/login"
        android:layout_marginTop="16dp"
        android:text="Log in" ... />
</LinearLayout>
```

The dots stand for layout_width and layout_height, which every view still needs. Use @string resources for real text.

ROWS INSIDE A COLUMN

Nesting LinearLayouts

```
LinearLayout (vertical)
    |-- TextView "Sign in"
    |-- LinearLayout (horizontal)
    | |-- Button "Cancel"
    | |-- Button "Log in"
```

Nesting is fine for simple screens. Each extra level adds work for Android, so use ConstraintLayout when a screen needs many levels.

PART 5 — RELATIVELAYOUT

Positioning by Relationship

In a RelativeLayout every child describes where it sits relative to the parent or to another child: below it, to the right of it, centred in the parent. Children can overlap.

Parent-relative
Align to the top, bottom, start or end of the layout

Sibling-relative
Place below, above or beside another view by its id

Overlap
Views can sit on top of each other

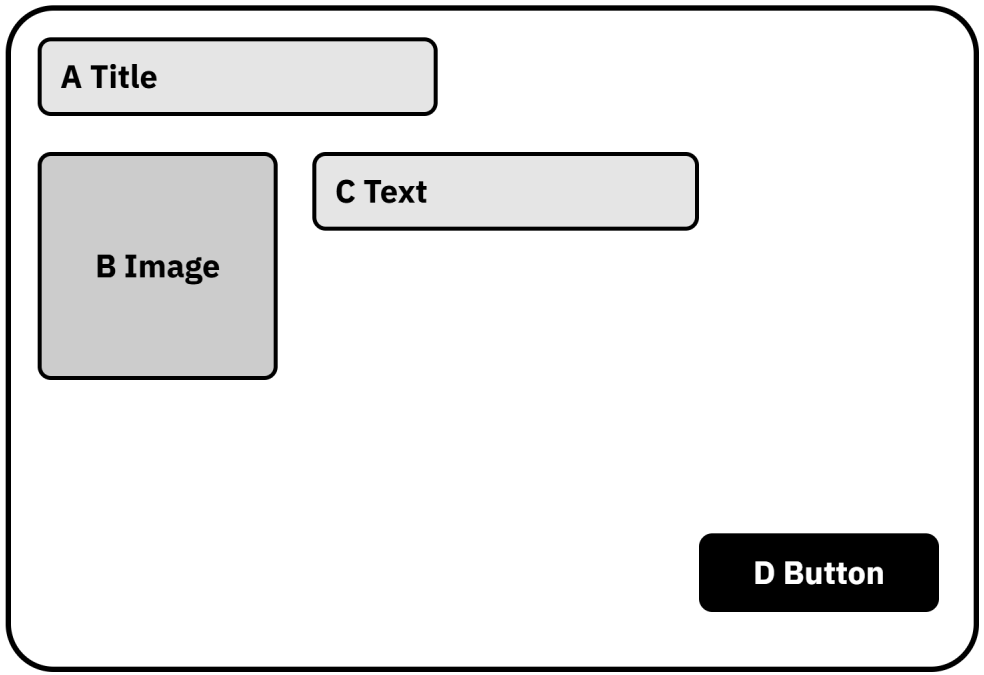
THE ATTRIBUTES TO LEARN

Key RelativeLayout Attributes

Attribute	Meaning	Value
`layout_alignParentTop`	Stick to the parent's top edge	true
`layout_alignParentEnd`	Stick to the parent's end edge	true
`layout_centerInParent`	Centre in the parent	true
`layout_below`	Place below another view	@id/title
`layout_toEndOf`	Place after another view	@id/icon
`layout_alignTop`	Match another view's top edge	@id/icon

READING THE RULES OFF A SCREEN

Relative Positions in a Picture



1. A: alignParentTop and alignParentStart
2. B: layout_below A
3. C: layout_toEndOf B, layout_alignTop B
4. D: alignParentBottom and alignParentEnd

TITLE ABOVE A FIELD

RelativeLayout in XML

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:text="Welcome" />
    <EditText android:id="@+id/email"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@id/title" />
</RelativeLayout>
```

VIEWS REFER TO EACH OTHER

Ids in a RelativeLayout

@+id/title
Creates a new id the first time it is written

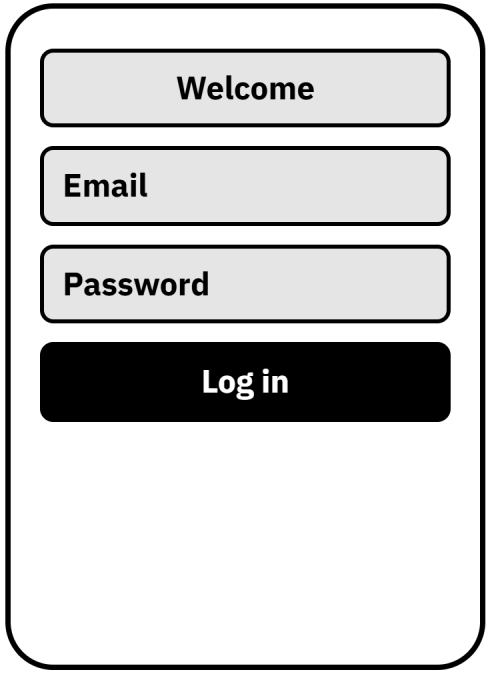
@id/title
Refers to an id that already exists

Circular rules
A below B and B below A is an error

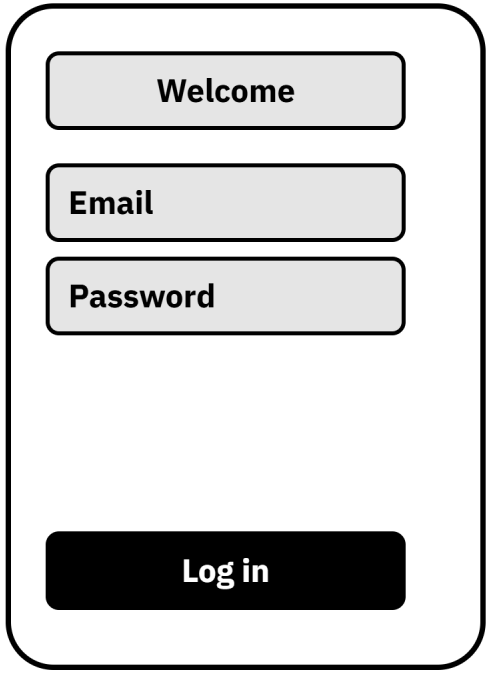
Every view you point at needs an android:id. Keep the chain of references short and easy to follow.

SAME FOUR VIEWS, DIFFERENT RULES

Login Screen: Side by Side



LinearLayout: views stack in XML order, top to bottom



RelativeLayout: each view is placed against the parent or a neighbour. Here the button is pinned to the bottom.

CHOOSING BETWEEN THEM

LinearLayout vs. RelativeLayout

	LinearLayout	RelativeLayout
Arrangement	One row or one column	Positions relative to others
Best for	Forms and button rows	Aligned or overlapping items
Watch out for	Deep nesting	Long chains of id references
Next step	Weights for sharing space	Replaced by ConstraintLayout in new code

LEARN FROM THESE NOW

Common Layout Mistakes

Missing width or height
Both attributes are required on every view

Using px
Use dp for sizes and sp for text

Weight with match_parent
Use 0dp in the direction that shares space

Text typed into the layout
Put strings in strings.xml and use @string

IN-CLASS EXERCISE, 15 MINUTES

Login Screen, Two Ways

1. Build a login screen with LinearLayout: title, email, password, button
2. Give the layout 24dp padding and the button a 16dp top margin
3. Add a Cancel and Log in row using layout_weight
4. Rebuild the same screen with RelativeLayout using layout_below
5. Compare both in the Design tab at two different screen sizes

BEFORE YOU LEAVE TODAY

Recap Checklist

1. Can you list what the manifest declares?
2. What is the difference between a normal and a dangerous permission?
3. What is the difference between a View and a ViewGroup?
4. When would you use dp, and when sp?
5. Can you place a view below another one in a RelativeLayout?

WHERE WE GO FROM HERE

Next Steps

Next theory
ConstraintLayout and XML layout files

Next lab
Implement all three layout types on the same screen

Before the lab, have your login screen from today's exercise saved and running.

Questions?

Manifest | Permissions | LinearLayout | RelativeLayout

Ekramul Haque Amin

Lecturer, AI & Data Science · Mobile Application and Development (0613-409)